

✓ Task 1: SQL Injection Attack and Mitigation

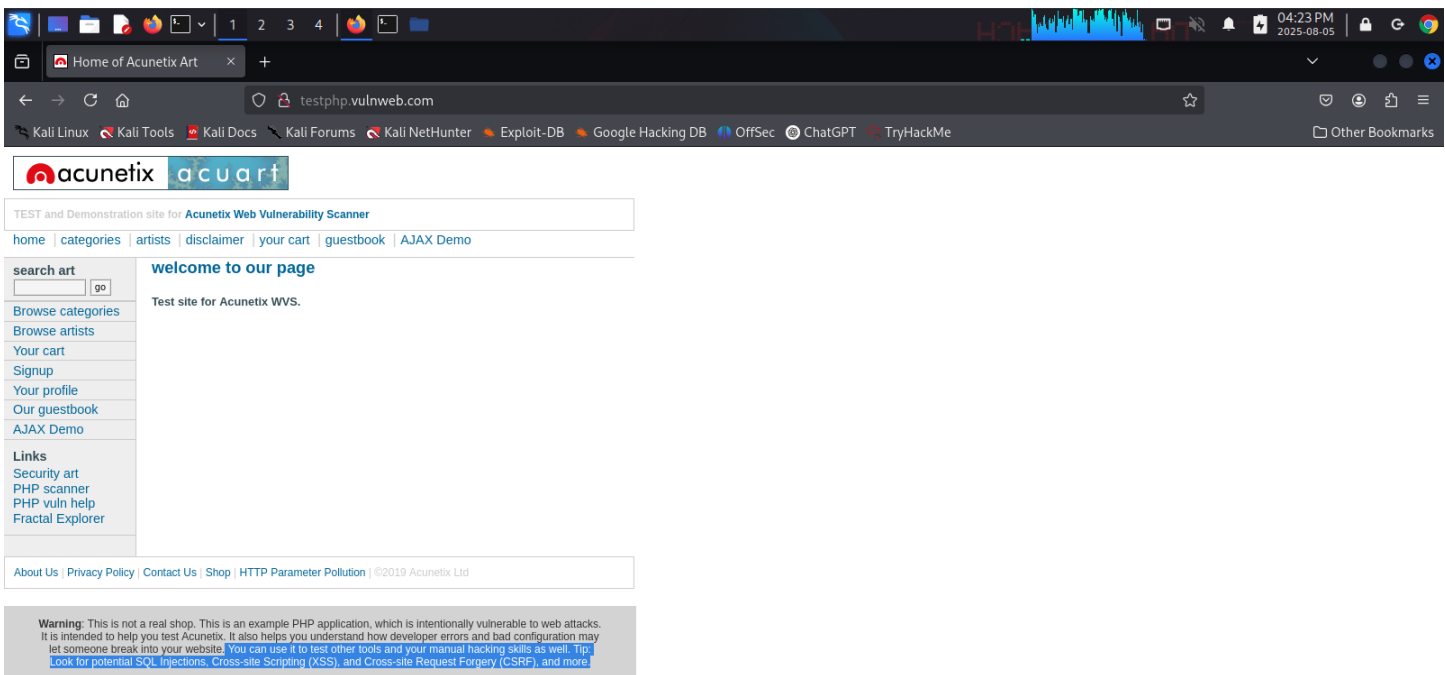
🎯 Objective:

- Demonstrate a SQL Injection (SQLi) attack on a test web application.
- Understand how it works.
- Show how to **mitigate** the vulnerability using secure coding.

🔧 Part 1: Performing the SQL Injection

✓ Target Website:

<http://testphp.vulnweb.com/>



🔍 Vulnerable Field:

Login Form <http://testphp.vulnweb.com/login.php>

💧 Test Input:

- **Username:** ' OR 1=1 --
- **Password:** anything

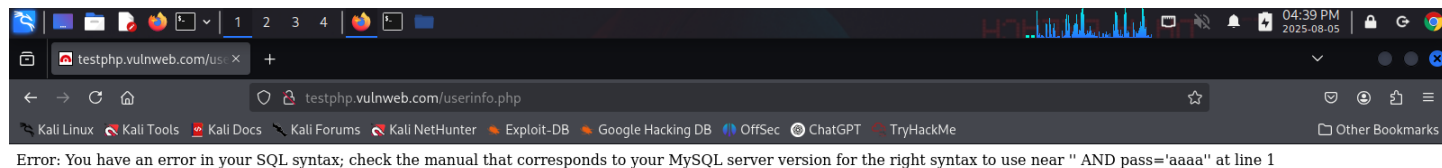
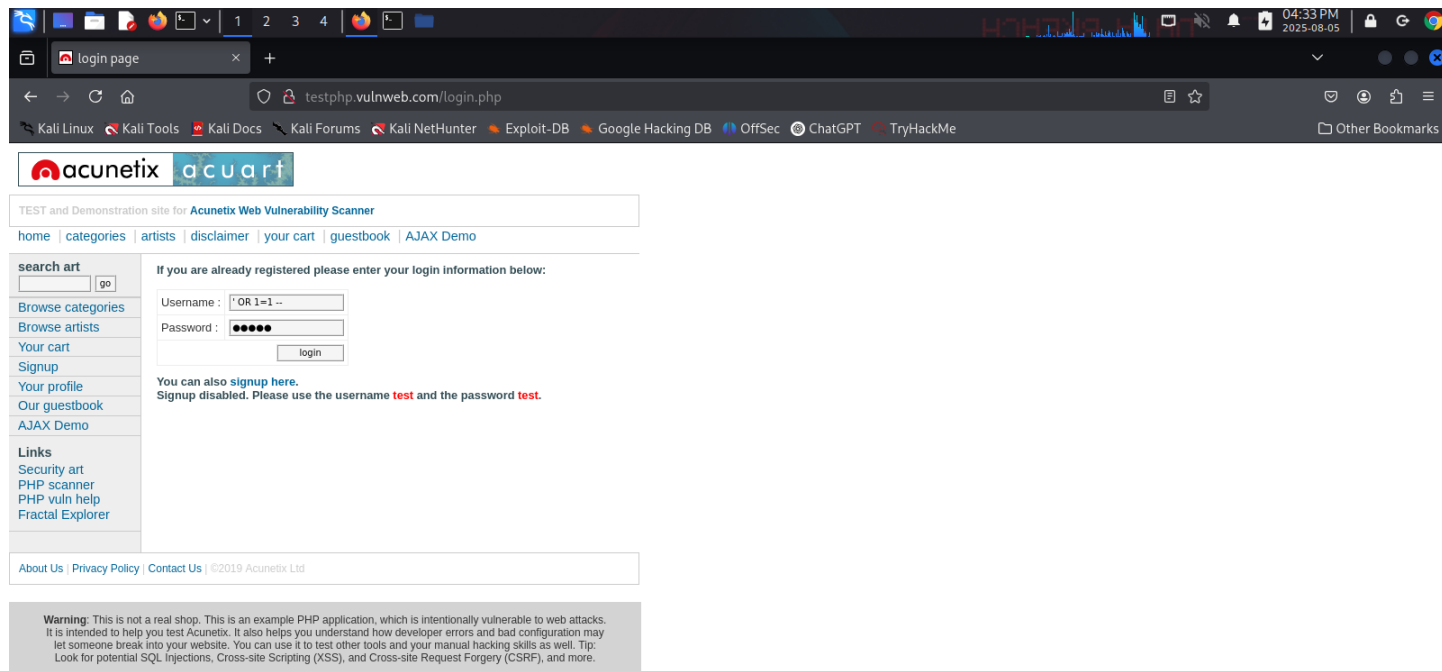
This will likely bypass login authentication if the backend query is vulnerable like:

```
SELECT * FROM users WHERE username = '$user' AND password = '$pass';
```

With the above payload, the query becomes:

```
SELECT * FROM users WHERE username = " OR '1'='1' AND password = 'anything';
```

Result: **Confirms SQL Vuln**



◆ Step 2: Attempted Login with Malicious Input

Injected a classic SQL payload into the **username** field to manipulate backend SQL logic.

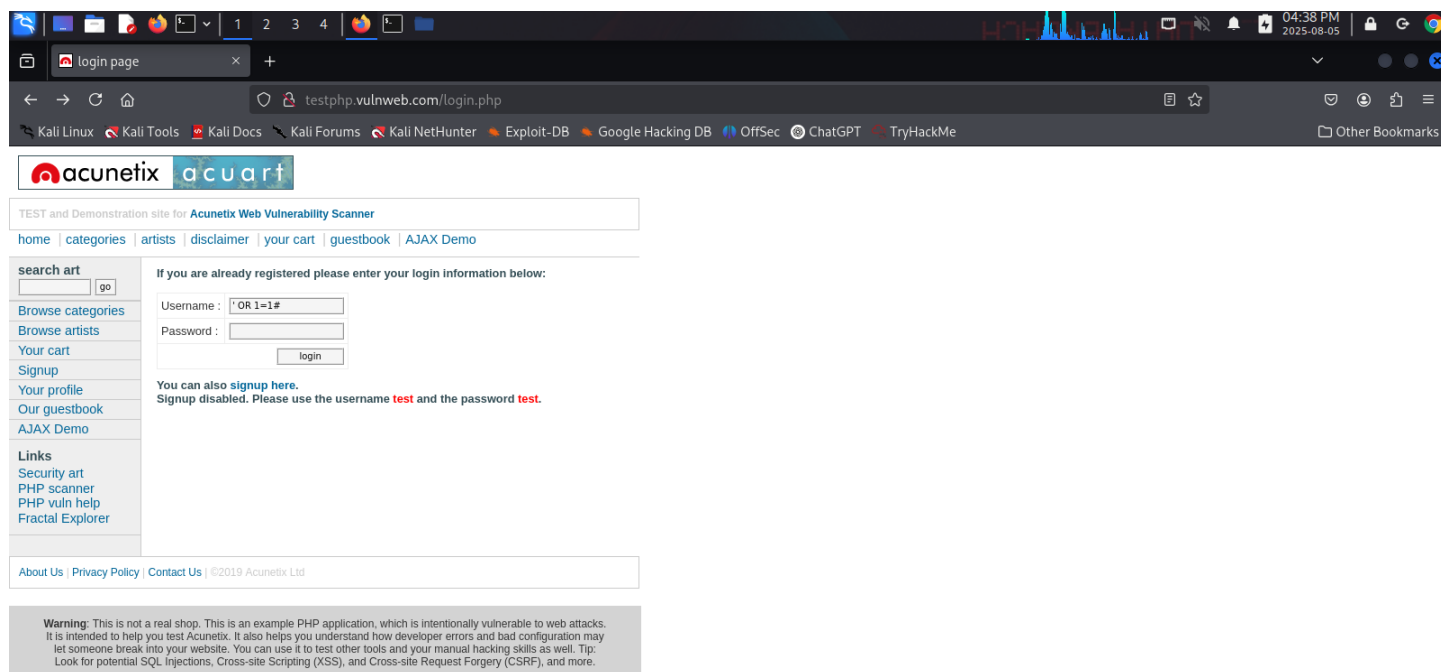
- **Payload Used:**

- ' OR 1=1#

- **Explanation:**

- ' closes the original query string
- OR 1=1 always returns **true**
- # comments out the rest of the SQL query

- **Password Field:** Left blank or filled with arbitrary data (ignored due to query manipulation)

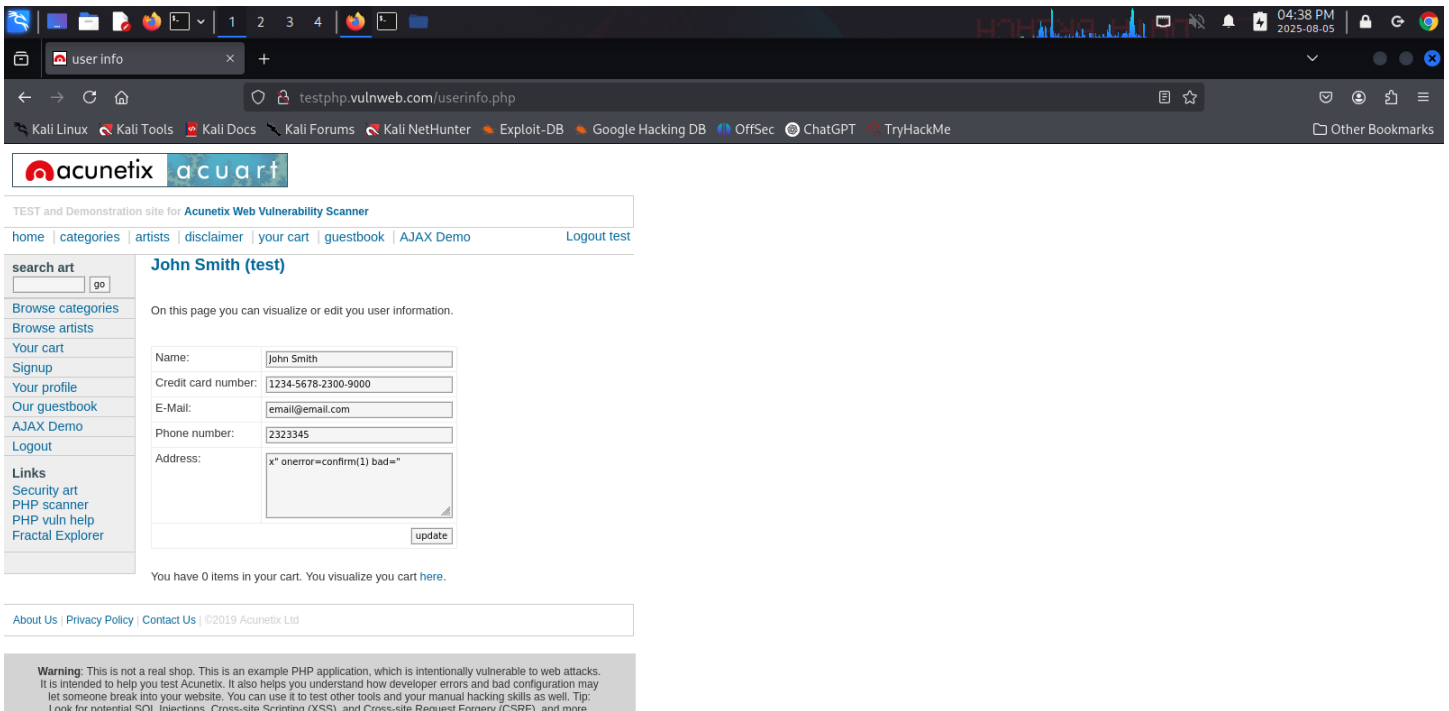


◆ Step 3: Observed Server Response

- The application **granted access** without valid credentials.
- This confirms **authentication bypass** via SQL injection.

✓ Result:

- 🗝️ **Login Bypass Successful**
- ! The server fails to sanitize input in the login form, allowing direct SQL injection.



Impact:

- Unauthorized access to protected user accounts
- Risk of privilege escalation
- Potential access to sensitive data

Recommended Fixes:

1. **Use Prepared Statements / Parameterized Queries**
 - e.g., use `mysqli_prepare()` or ORM query binding.
2. **Input Validation & Sanitization**
 - Apply whitelisting where applicable.
3. **Error Message Suppression**
 - Do not expose SQL errors to users.
4. **Web Application Firewall (WAF)**
 - Add an extra layer of filtering against malicious inputs.

Explanation of How SQLi Works

- **SQL Injection** happens when user inputs are directly inserted into SQL queries without sanitization or validation.
 - Malicious input can **manipulate** the query logic.
 - Common outcomes:
 - Login bypass
 - Database dumping
 - Data leakage
 - Full database access
-

Part 2: Mitigation / Fix

Fix Strategy:

Use **Parameterized Queries (Prepared Statements)** to prevent injection.

Example Fix in PHP (with MySQLi):

Vulnerable Code:

```
$query = "SELECT * FROM users WHERE username = '$user' AND password = '$pass'";  
$result = mysqli_query($conn, $query);
```

Secure Code:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");  
$stmt->bind_param("ss", $user, $pass);  
$stmt->execute();  
$result = $stmt->get_result();
```

Additional Defenses:

- Input validation & sanitization
- Minimum privilege for DB users
- Web Application Firewalls (WAF)
- Error handling (don't expose SQL errors)